

AI Use-Case Intake & Prioritization Console

A single front door for an AI opportunity pipeline: structured intake, rubric scoring, decision briefs.

A practical walkthrough of what the console does and how to use every screen. If you just want to get it running, jump to [Getting started](#); if you want to understand the scoring, read [Core concepts](#).

Synthetic data only. Everything in this tool is made-up, authored-for-demonstration data. There is no real claims, member, or PHI data anywhere in it, and it is not connected to any production system.

Table of contents

1. [What it is and who it's for](#)
 2. [Getting started](#)
 3. [Core concepts](#)
 4. [Screen-by-screen walkthrough - The portfolio - Submitting a use case - The use-case page & scoring - Overriding scores - The quadrant - The decision brief - The evaluation report](#)
 5. [Choosing a model provider](#)
 6. [Resetting for a clean demo](#)
 7. [Troubleshooting & FAQ](#)
 8. [Glossary](#)
-

What it is and who it's for

Organizations adopting AI quickly accumulate more ideas than they can act on, with no consistent way to compare them. This console is a **single front door** for an AI opportunity pipeline: a structured way to capture proposed use cases, score them on a consistent rubric, and produce a prioritized portfolio plus a one-page decision brief per idea — including an explicit judgment on whether AI is even the right tool.

It's for anyone triaging where to apply AI — an innovation or transformation function, a data/AI team, or a business unit. You bring the ideas; the console helps you compare them on evidence and record *why* each one was greenlit, parked, or rejected.

The core idea: the AI *proposes* scores and a recommendation; **a human reviews and decides**. The tool is a decision aid, never the decision-maker.

Getting started

You need **Python 3.11+**. From the project folder:

```
make install      # create a virtual environment and install dependencies
cp .env.example .env # then edit .env (see below)
make reset       # load the 10 synthetic seed use cases
make run         # start the server
# open http://localhost:8000
```

If you don't have `make`, the equivalent raw commands are in the project [README.md](#).

Setting up scoring

Open `.env` and choose how scoring runs:

- **Live Claude model (recommended):** set `ANTHROPIC_API_KEY=...` and leave `MODEL_PROVIDER=anthropic`. This is the real experience.
- **Fully offline (no key, no network):** set `MODEL_PROVIDER=stub`. Scoring then uses a deterministic heuristic — useful for demos on a laptop with no connectivity. The UI clearly labels stub output as heuristic.
- **Local model:** set `MODEL_PROVIDER=ollama` (with Ollama running locally) for a privacy-preserving, on-prem-style path.

You can switch providers any time by editing `.env` and restarting `make run`. See [Choosing a model provider](#) for the full reference.

Once running, the home page shows a **portfolio of 10 seeded use cases** — eight healthcare- payer "fit" examples and three deliberate "not a fit for AI" examples.

Core concepts

The five scoring dimensions

Every use case is scored on five dimensions, each an integer **1–5 where 5 is the most favorable** for prioritization:

Dimension	What it measures
Impact	Business / member value if the use case works well.
Feasibility	How achievable it is with current AI and the data you actually have.
Risk	Risk posture — 5 = low / well-managed risk , 1 = severe risk (regulatory, clinical, fairness).
Adoption	How readily the stakeholders would actually use it.
Strategic Value	Alignment with the organization's AI-transformation agenda.

The Risk scale is inverted on purpose. So that "higher is always better," a high Risk score means *low, well-managed* risk. A use case that scores **Risk = 1** is **highly risky**. Each Risk rationale states the posture in words, and every scoring view shows this reminder, so you're never guessing which way the number points.

Each dimension comes with a **one-line rationale** — the console never shows an unexplained number.

Composite priority score (0–100)

The five dimensions combine into a single **priority score from 0 to 100**, a weighted mean:

Weight	Dimension
30%	Impact
25%	Strategic Value
20%	Feasibility
15%	Adoption
10%	Risk

Higher = higher priority. The weights are deliberate (Impact and Strategic Value lead) and are a single setting in the code if your organization prioritizes differently.

The "fit / not a fit for AI" guardrail

Beyond scoring, the console judges whether **AI is even the right tool**. A use case is flagged **not a fit for AI** when it's better solved *without* AI — for example:

- **Deterministic computation** (e.g. reconciling a ledger): a rules engine or SQL is exact, auditable, and cheaper than a probabilistic model.
- **A simple lookup or integration** (e.g. checking eligibility against the system of record): that's an API call, not a prediction problem.
- **A fully-autonomous high-stakes decision** that legally or ethically requires a human (e.g. issuing a coverage denial): AI may *support* the review, but must not decide alone.

Not-a-fit cases carry a written reason and are **grouped separately** in the portfolio so they can't out-rank real opportunities. Knowing where *not* to use AI is treated as a feature.

Human-in-the-loop and provenance

The model **proposes**; you **decide**. Every score, rationale, ROI hypothesis, and fit flag is editable. When you override a value, the console stores it as a **new record** (it never overwrites history) tagged with its **source** — **seed** (authored), **llm** (model-proposed), or **human** (your override) — along with the model id and prompt version. You always have an audit trail of what changed and who changed it.

Screen-by-screen walkthrough

1. The portfolio (home)

The landing page at  is your prioritized pipeline.

- **Ranked by composite priority.** AI-fit cases appear first, highest priority at the top. A **Priority** pill (colored by tier) shows the 0–100 score; **Impact** and **Feasibility** are called out as their own columns.
- **"Not a fit for AI" cases are grouped below a divider** — visible for transparency, but ranked separately so they never push a real opportunity down the list.

- **AI-fit badge** on each row: *AI fit*, *Not a fit*, or *Unscored*.
- **Search** filters the list as you type (title, problem, workflow, submitter).
- **Sort** by clicking the **Use case / Priority / Impact / Feasibility** column headers; click again to reverse direction.
- Each row links to the use case (its title) and to its **Brief**.

From here you can also jump to **Quadrant** or **+ New use case** in the top nav.

2. Submitting a use case

Click **+ New use case** (top nav or the portfolio button) to open the intake form. Capture the opportunity in plain language:

Field	Notes
Title (<i>required</i>)	A short, recognizable name.
Problem statement (<i>required</i>)	What hurts today, in plain language. This is the most important field — it's what the model reasons about.
Affected workflow	Where in the business this lives.
Data availability	What data exists to support it.
Stakeholders	Who's involved or affected.
Current pain	The cost of the status quo (time, errors, delays).
Submitter	Who proposed it.

Only **Title** and **Problem** are required; the rest sharpen the scoring. Click **Save use case** and you'll land on the new use case's page, ready to score. (You can come back and **Edit** any of these fields later.)

Tip for a good problem statement: describe the pain and the workflow, not a solution. "Nurses manually read every prior-auth request, creating a 3-day backlog" scores better than "We should build an AI for prior auth."

3. The use-case page & scoring

Open any use case to see two cards:

- **Intake** — the captured fields.
- **Scoring** — initially "Not scored yet" for cases you create (seeded cases arrive pre-scored).

Click **Run AI scoring**. The model reads the intake and proposes:

- the five dimension scores, **each with a one-line rationale**,
- an **ROI hypothesis**,
- a **fit / not a fit** judgment with a reason,
- and the console computes the **composite priority score** and **quadrant** placement.

A scale legend reminds you that 5 is most favorable (and that Risk = 5 means low risk). At the bottom, a **provenance line** records the source, model, prompt version, and timestamp — so the score is always traceable. Use **Re-run AI scoring** to score again (each run is stored as new history).

If scoring can't run (e.g. `MODEL_PROVIDER=anthropic` but no API key), the card shows a clear message telling you what to fix — it never invents numbers.

4. Overriding scores (human-in-the-loop)

You are the decision-maker. Click **Override** on the scoring card to open an editable form:

- adjust any of the five **1–5 scores**,
- rewrite any **rationale**,
- edit the **ROI hypothesis**,
- toggle the **"fit for AI"** checkbox and edit its reason.

Click **Save override**. The card returns with a **Human override** badge, and your values become the current score. The previous (model or seed) score isn't deleted — it stays in the history. This is the "AI proposes, a human decides" pattern made concrete: useful in any regulated or high-stakes setting where you need to show *who* set a value and *why*.

5. The quadrant

Click **Quadrant** in the nav for the classic **Impact × Feasibility** view of every scored case:

- **Vertical axis = Impact, horizontal axis = Feasibility**, each 1–5.
- **Filled blue dots = AI fit; hollow amber dots = not a fit**. Hover any dot for the title and its scores.
- The four cells:

	Low feasibility	High feasibility
High impact	Big bets — valuable but hard	Quick wins — do these now
Low impact	Deprioritize	Incremental — easy but minor

Click a dot to open that case's decision brief. This is the one-glance picture for a portfolio conversation: what to do now (top-right), what to invest in (top-left), and what to set aside.

6. The decision brief

From a use case's scoring card (or any quadrant dot, or the portfolio's **Brief** link), open the **Decision brief** — a clean, one-page summary suitable for an executive audience:

- the **verdict** up top: priority score, quadrant, and AI-fit badge (with the "why not AI" callout for not-a-fit cases),
- **problem & context**,
- the **five scores with rationales**,
- the **ROI hypothesis**,
- and the **provenance** footer.

Click **Print / Save as PDF** to produce the artifact — the app chrome (nav, banners) is automatically stripped for a clean printout. This is the document you'd attach to a greenlight/park/reject decision.

7. The evaluation report

Scoring quality is checked, not assumed. From a terminal:

```
make eval
```

This re-scores the seed set and prints a short report on three things:

- **Stability** — re-scoring produces consistent composite scores (low spread).
- **Guardrail** — the deliberate "not a fit for AI" cases are flagged correctly (recall).
- **Rubric adherence** — every score carries a rationale per dimension and an ROI line.

Example:

```
Provider: claude-opus-4-8   Runs per case: 3
Stability (composite spread across runs):      mean 1.4   max 4.0
Guardrail ('not a fit for AI' cases flagged):  recall 3/3 (100%)   missed: none
Rubric adherence (rationale per dimension + ROI): pass 30/30 (100%)
Overall: PASS
```

It runs **offline by default** (falls back to the deterministic stub if no provider/key is configured), and the report always names the provider that produced the numbers. With the stub, stability is perfectly deterministic; with a live model, you'll see real run-to-run variance — which is exactly what the stability check is there to surface.

Choosing a model provider

Set `MODEL_PROVIDER` in `.env`:

Value	Behavior	When to use
<code>anthropic</code> (default)	Scores with the Claude model in <code>ANTHROPIC_MODEL</code> (default <code>claude-opus-4-8</code>). Requires <code>ANTHROPIC_API_KEY</code> .	The real experience; live demos with connectivity.
<code>ollama</code>	Scores with a local model via Ollama (<code>OLLAMA_HOST</code> , <code>OLLAMA_MODEL</code>).	Privacy-preserving / on-prem-style; no data leaves the machine.
<code>stub</code>	Deterministic heuristic, no network or key. Output is clearly labelled as heuristic.	Offline demos, CI, exploring the UI without a key.

Full `.env` reference:

```
MODEL_PROVIDER=anthropic      # anthropic | ollama | stub
ANTHROPIC_API_KEY=            # required for the anthropic provider
ANTHROPIC_MODEL=claude-opus-4-8 # which Claude model to score with
OLLAMA_HOST=http://localhost:11434 # used when MODEL_PROVIDER=ollama
OLLAMA_MODEL=llama3.1
DATABASE_PATH=data/console.db  # SQLite file location
```

The `.env` file is never committed (it's git-ignored). Keep your API key out of source control.

Resetting for a clean demo

`make reset` deletes the database and re-loads the 10 seed use cases, returning the app to a known, clean state. Use it before a demo, or any time you want to start over:

```
make reset # clean slate, 10 seeded cases (3 deliberate "not a fit")
```

Two related commands:

- `make seed` — idempotent and **non-destructive to your work**: it refreshes the authored seed cases but leaves any LLM or human-override scores you've added in place.
- `make reset` — the clean-room option: it **deletes the database file first**, then re-seeds, so you get a pristine demo state with no leftover scores or added cases.

Use `make reset` before a demo; use `make seed` if you just want to (re)load the seed without discarding your own scoring work.

There's a tight, practiced demo script in `DEMO.md` if you want a 75-second walkthrough.

Troubleshooting & FAQ

"Run AI scoring" shows an error about `ANTHROPIC_API_KEY`. You're on the `anthropic` provider with no key. Either add `ANTHROPIC_API_KEY` to `.env`, or set `MODEL_PROVIDER=stub` for offline scoring. Restart `make run` after editing `.env`.

Why is a Risk score of 5 a good thing? By design, every dimension uses "higher is better." For Risk that means 5 = low / well-managed risk. The rationale always describes the posture in words so it's unambiguous.

A "not a fit" case has decent scores — why isn't it ranked higher? Not-a-fit cases are grouped separately at the bottom of the portfolio regardless of their composite, so they can never out-rank a genuine opportunity. The guardrail is intentional.

I overrode a score — did I lose the model's original? No. Overrides are stored as new records; the prior score remains in history. The current value is just the most recent one.

Does my data leave my machine? Only the `anthropic` provider sends the use-case text to the Claude API for scoring. `ollama` and `stub` run entirely locally. All seed data is synthetic regardless.

`make` isn't available on my machine. The `README.md` lists the equivalent raw `python / uvicorn` commands.

Can I change the priority weights or the rubric? Yes — the weights live in one place (`app/prioritization.py`) and the scoring rubric in the system prompt (`app/scoring.py`). Both are documented in `DECISIONS.md`.

Glossary

- **Use case** — a proposed application of AI captured through intake.
- **Composite priority score** — the 0–100 weighted-mean ranking value.
- **Dimension** — one of the five scored axes (Impact, Feasibility, Risk, Adoption, Strategic Value).

- **AI fit / not a fit** — the judgment of whether AI is the right tool for this problem.
 - **Quadrant** — the Impact × Feasibility 2×2 (Quick wins, Big bets, Incremental, Deprioritize).
 - **Decision brief** — the printable one-page summary for one use case.
 - **Provider** — the scoring backend (`anthropic` , `ollama` , or `stub`).
 - **Provenance** — the source (`seed` / `llm` / `human`), model, and prompt version recorded with each score.
 - **Override** — a human-edited score, stored as a new record without deleting history.
-

This is a portfolio prototype built on synthetic data. For the technical architecture and the rationale behind each design choice, see `README.md` and `DECISIONS.md` .